Coding & Solution

Date	29 June 2026
Team ID	xxxxxx
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented OptiCrop application. The project demonstrates a fully functional Machine Learning-based crop recommendation system developed using Python and Flask. The code follows modular programming principles, meaningful naming conventions, proper documentation, and good coding practices to ensure maintainability and scalability.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/sivaganesh7/Optic-Crop
Programming Language(s)	Python, HTML5, CSS3, JavaScript
Framework(s) Used	Flask, Scikit-learn, Pandas, NumPy
Key Features Implemented	Crop recommendation using Machine Learning, soil and weather parameter analysis, Flask web application, input validation, responsive user interface, trained model integration
Pending / Incomplete Features	Fertilizer recommendation, weather API integration, crop disease detection, yield prediction, mobile application

Setup / Run Instructions	➢ Clone the project repository from GitHub.
	➢ Install the required dependencies using:
	`pip install -r requirements.txt`
	➢ Navigate to the Flask web application folder.
	➢ Run the application using:
	`python app.py`
	➢ Open a web browser and visit:
	`http://127.0.0.1:5000`
	➢ Enter the required soil and weather parameters.
	➢ Click the **Predict** button to view the recommended crop

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

- The project follows a modular architecture with separate folders for data analysis, preprocessing, model building, testing, documentation, and Flask web application.
- Input validation is implemented on both the client side (JavaScript) and server side (Flask).
- The trained Machine Learning model is integrated using Joblib for efficient prediction.
- The code follows Python coding standards with meaningful variable names and reusable functions.
- The application is designed for future enhancements such as fertilizer recommendation, weather API integration, crop disease detection, and yield prediction.